CURSO 1: LÓGICA DE PROGRAMAÇÃO APLICADA A DADOS

Apostila Prática para Impressão — Metodologia Iterativa e Incremental com Refatoração

Ementa Completa do Curso (5 Aulas Práticas)

- **Aula 1:** Preparação do Ambiente, Variáveis, Entrada/Saída e Refatoração Matemática (Cálculo de Indicadores).
- **Aula 2:** Estruturas Condicionais (`if/elif/else`) e Regras de Negócio (Classificação e Validação de Dados).
- **Aula 3:** Laços de Repetição (`while` e `for`) e Processamento em Lote (Limpeza de Registros).
- **Aula 4:** Vetores e Matrizes na Prática (Manipulação de Listas de Dados e Filtros).
- **Aula 5:** Modularização e Projeto Final (Simulador de Relatório de Vendas).

AULA 1: AMBIENTE PROFISSIONAL E LÓGICA SEQUENCIAL

1. Preparação do Ambiente de Trabalho

Antes de escrever qualquer linha de lógica, todo engenheiro de software isola seu projeto. Abra o terminal do seu computador e digite os comandos abaixo passo a passo:

```
# 1. Crie a pasta do curso e entre nela
mkdir curso_logica_dados
cd curso_logica_dados

# 2. Crie o ambiente virtual (venv)
python -m venv venv

# 3. Ative o ambiente virtual
# No Windows:
venv\Scripts ctivate
# No Linux/Mac:
source venv/bin/activate

# 4. Crie o arquivo de dependências vazio por enquanto (padrão de projetos)
pip freeze > requirements.txt
```

> **Nota Técnica:** Mesmo sem bibliotecas externas nesta primeira aula, manter o `venv` ativo e o `requirements.txt` criado prepara a estrutura profissional para quando formos importar pacotes de análise de dados.

2. O Problema de Negócio

Você recebeu a tarefa de criar o módulo inicial de um sistema de gestão escolar (similar ao que será usado futuramente no banco de dados do Django). O sistema precisa receber o nome de um aluno e duas notas de avaliações, calcular a média e exibir um resumo formatado.

3. Código Inicial (Versão 1.0 - O Modelo Base)

Crie um arquivo chamado `aula01_notas.py`. Digite o código abaixo exatamente como está, execute no terminal (`python aula01_notas.py`) e observe o resultado.

```
# aula01_notas.py - Versão 1.0 (Modelo Inicial)

# Entrada de dados fixa (hardcoded) para testar o fluxo
nome_aluno = "Carlos Silva"
nota1 = 8.0
nota2 = 6.0

# Processamento lógico sequencial
soma = nota1 + nota2
media = soma / 2

# Saída de dados
print("Aluno:", nome_aluno)
print("Média Final:", media)
```

4. Iteração e Incremento (Versão 2.0)

Um código estático não serve para sistemas reais. Vamos **iterar** sobre o modelo anterior e **incrementar** duas novas funcionalidades:

1. Permitir que o usuário digite os dados via teclado (`input`).
2. Adicionar um peso diferente para cada avaliação (Média Ponderada: Prova 1 tem peso 4, Prova 2 tem peso 6).

Altere o seu arquivo `aula01_notas.py` para ficar assim:

```
# aula01_notas.py - Versão 2.0 (Iteração e Incremento)

print("--- SISTEMA DE LANÇAMENTO DE NOTAS ---")

# Incremento 1: Entrada dinâmica de dados
nome_aluno = input("Digite o nome do aluno: ")
nota1_texto = input("Digite a nota da Prova 1: ")
nota2_texto = input("Digite a nota da Prova 2: ")

# Conversão de tipos (Casting) necessária para cálculos
nota1 = float(nota1_texto)
nota2 = float(nota2_texto)

# Incremento 2: Nova regra de negócio (Média Ponderada)
peso1 = 4
peso2 = 6
soma_pesos = peso1 + peso2

nota1_ponderada = nota1 * peso1
nota2_ponderada = nota2 * peso2
soma_total = nota1_ponderada + nota2_ponderada
media_final = soma_total / soma_pesos

# Saída de dados
print("---------------------------------------")
print("Aluno:", nome_aluno)
print("Nota 1 (Peso 4):", nota1)
print("Nota 2 (Peso 6):", nota2)
print("Média Ponderada:", media_final)
```

5. Refatoração e Eficiência (Versão 3.0)

Na Engenharia de Software, **refatorar** significa melhorar a estrutura interna do código (torná-lo mais limpo, legível e rápido) **sem alterar o comportamento externo** para o usuário.

Observe como o código da V2.0 tem variáveis intermediárias desnecessárias e `prints` repetitivos. Vamos substituir esses trechos usando boas práticas do Python (`f-strings`, conversão direta e constantes em maiúsculo).

```
# aula01_notas.py - Versão 3.0 (Código Refatorado)

# Constantes de regra de negócio (fáceis de achar e alterar no futuro)
PESO_PROVA_1 = 4
PESO_PROVA_2 = 6

print("--- SISTEMA DE LANÇAMENTO DE NOTAS ---")

# Entrada e conversão direta na mesma linha
nome_aluno = input("Digite o nome do aluno: ").strip().title()
nota1 = float(input("Digite a nota da Prova 1: "))
nota2 = float(input("Digite a nota da Prova 2: "))

# Cálculo direto e limpo
media_final = ((nota1 * PESO_PROVA_1) + (nota2 * PESO_PROVA_2)) /
(PESO_PROVA_1 + PESO_PROVA_2)

# Saída formatada profissionalmente com f-string e limite de 2 casas decimais
print("-" * 38)
print(f"Aluno: {nome_aluno}")
print(f"Notas: P1 = {nota1:.1f} | P2 = {nota2:.1f}")
print(f"Média Ponderada Final: {media_final:.2f}")
```

O que melhoramos mantendo a mesma função?

- `.strip().title()` limpa espaços acidentais e padroniza o nome (ex: `" ana souza "` vira `"Ana Souza"`), prática essencial na limpeza de dados.
- Eliminamos 5 variáveis temporárias de memória.
- Formatamos a saída numérica (`:.2f`) para evitar dízimas periódicas na tela.

6. Exercício de Fixação

Crie um novo arquivo chamado `exercicio01_vendas.py` e aplique o mesmo ciclo (comece simples e refatore) para resolver o seguinte problema:

- **Cenário:** Você está criando a lógica de fechamento de caixa de uma loja.
- **Requisitos:**
  1. Receba o nome do produto, o valor unitário e a quantidade vendida.
  2. Calcule o valor bruto da venda.
  3. Aplique um imposto fixo de **12%** sobre o valor bruto para descobrir o valor líquido que fica para a loja.
  4. Exiba o resumo formatado com duas casas decimais.

Gabarito Comentado (Versão Refatorada)

```
# exercicio01_vendas.py - Solução Refatorada

TAXA_IMPOSTO = 0.12  # 12% em formato decimal

print("--- FECHAMENTO DE VENDA ---")
produto = input("Nome do produto: ").strip().capitalize()
preco_unitario = float(input("Preço unitário (R$): "))
quantidade = int(input("Quantidade vendida: "))

valor_bruto = preco_unitario * quantidade
valor_imposto = valor_bruto * TAXA_IMPOSTO
valor_liquido = valor_bruto - valor_imposto

print("-" * 27)
print(f"Produto: {produto} ({quantidade}x)")
print(f"Total Bruto: R$ {valor_bruto:.2f}")
print(f"Impostos (12%): R$ {valor_imposto:.2f}")
print(f"Valor Líquido: R$ {valor_liquido:.2f}")
```